МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ІВАНА ФРАНКА

Факультет електроніки та комп’ютерних технологій

З ЛАБОРАТОРНОЇ РОБОТИ №2
з дисципліни "Об'єктно орієнтоване програмування (C++)"
на тему " Реалізація алгоритмів у функціональному стилі"

Виконала:

студентка 1 курсу

групи ФеП-12
Ступницька Юля Ярославівна

Перевірив:

асистент

Левуш Павло Назарович

Львів – 2026

Мета роботи

Ознайомитися з реалізацією алгоритмів у функціональному стилі в мові C++, та створити програму для перевірки належності точки трикутнику різними методами.

Хід роботи

Крок 1. Опис структури та заголовкового файлу (triangle.h) Описано структури Point і Triangle та оголошено функції.

```
triangle.h > ...
#ifndef TRIANGLE_H
#define TRIANGLE_H

Windsurf: Refactor | Explain
struct Point {
    double x, y;
};

Windsurf: Refactor | Explain
struct Triangle {
    Point A, B, C;

    double area() const;
    bool isDegenerate() const;

    int checkByArea(const Point &P) const;   // 0 - всередині, 1 - ззовні, 2 - межа
    int checkByVector(const Point &P) const;
};

double distance(const Point &p1, const Point &p2);
double cross(const Point &A, const Point &B, const Point &P);

#endif // TRIANGLE_H
```

Крок 2. Реалізація функцій (triangle.cpp)

Було реалізовано:

- функцію `distance()` — для обчислення відстані між точками
- функцію `cross()` — векторний добуток
- метод `area()` — площа за формулою Герона
- метод `isDegenerate()` — перевірка виродженого трикутника

Використовується знак векторного добутку для визначення положення точки:

- однакові знаки → точка всередині
- різні знаки → точка ззовні
- нуль → точка на межі

```cpp
#include "triangle.h"
#include <cmath>

Windsurf: Refactor | Explain | Generate Function Comment | ×
double distance(const Point &p1, const Point &p2) {
    return sqrt(pow(p2.x - p1.x, 2) + pow(p2.y - p1.y, 2));
}

Windsurf: Refactor | Explain | Generate Function Comment | ×
double cross(const Point &A, const Point &B, const Point &P) {
    return (B.x - A.x) * (P.y - A.y) -
           (B.y - A.y) * (P.x - A.x);
}

// Герон
Windsurf: Refactor | Explain | ×
double Triangle::area() const {
    double a = distance(A, B);
    double b = distance(B, C);
    double c = distance(C, A);

    double s = (a + b + c) / 2;
    double val = s * (s - a) * (s - b) * (s - c);

    if (val < 0) val = 0;

    return sqrt(val);
}

Windsurf: Refactor | Explain | Generate Function Comment | ×
bool Triangle::isDegenerate() const {
    return area() < 1e-9;
}

// метод площі
Windsurf: Refactor | Explain | ×
int Triangle::checkByArea(const Point &P) const {
    const double eps = 1e-9;

    if (isDegenerate()) return 1;

    Triangle t1{A, B, P};
    Triangle t2{B, C, P};
    Triangle t3{C, A, P};

    double S = area();
    double Ssum = t1.area() + t2.area() + t3.area();

    if (fabs(S - Ssum) < eps) {
        if (t1.area() < eps || t2.area() < eps || t3.area() < eps)
            return 2;
        return 0;
    }

    return 1;
}

// векторний метод
Windsurf: Refactor | Explain | ×
int Triangle::checkByVector(const Point &P) const {
    const double eps = 1e-9;

    double c1 = cross(A, B, P);
    double c2 = cross(B, C, P);
    double c3 = cross(C, A, P);

    if (fabs(c1) < eps || fabs(c2) < eps || fabs(c3) < eps)
        return 2;

    if ((c1 > 0 && c2 > 0 && c3 > 0) ||
        (c1 < 0 && c2 < 0 && c3 < 0))
        return 0;
    return 1;
}
```

Крок 3. Основна програма (main.cpp)

Реалізовано введення даних, перевірку та виведення результатів.

```
main.cpp > main()
#include <iostream>
#include <limits>
#include <windows.h>
#include "triangle.h"

using namespace std;

Windsurf Refactor | Explain | Generate Function Comment | X
bool inputPoint(Point &p) {
    if (!(cin >> p.x >> p.y)) {
        cin.clear();
        cin.ignore(numeric_limits<streamsize>::max(), '\n');
        return false;
    }
    return true;
}

Windsurf Refactor | Explain | Generate Function Comment | X
int main() {
    SetConsoleOutputCP(65001);
    SetConsoleCP(65001);

    Triangle t;

    cout << "Введіть координати A (x y): ";
    if (!inputPoint(t.A)) {
        cout << "Помилка вводу\n";
        return 1;
    }

    cout << "Введіть координати B (x y): ";
    if (!inputPoint(t.B)) {
        cout << "Помилка вводу\n";
        return 1;
    }

    cout << "Введіть координати C (x y): ";
    if (!inputPoint(t.C)) {
        cout << "Помилка вводу\n";
        return 1;
    }

    if (t.isDegenerate()) {
        cout << "Трикутник вироджений (площа = 0)\n";
    } else {
    cout << "Трикутник коректний\n";
    }

    cout << "\nВибраний трикутник з вершинами: { " << t.A.x << " , " << t.A.y
        << " },  { " << t.B.x << " , " << t.B.y << " },  { " << t.C.x << " , "
        << t.C.y << " }\n"
        << endl;

    while (true) {
        Point p;

        cout << "\nВведіть точку (x y) або будь-що для виходу: ";

        if (!inputPoint(p)) {
            cout << "Завершення програми\n";
            break;
        }

        int resArea = t.checkByArea(p);
        int resVector = t.checkByVector(p);

        cout << "Точка (" << p.x << ", " << p.y << "):\n";

        // площа
        if (resArea == 0) cout << "  Площа: всередині\n";
        else if (resArea == 1) cout << "  Площа: зовні\n";
        else cout << "  Площа: на межі\n";

        // вектор
        if (resVector == 0) cout << "  Вектор: всередині\n";
        else if (resVector == 1) cout << "  Вектор: зовні\n";
        else cout << "  Вектор: на межі\n";
    }

    return 0;
}
```

Крок 4. Результат виконання

Програма коректно визначає положення точки відносно трикутника.

```
PS D:\university\1\2\OOP\lab2> .\main.exe
Введіть координати A (x y): a
Помилка вводу
```

```
PS D:\university\1\2\OOP\lab2> .\main.exe
Введіть координати A (x y): 0 0
Введіть координати B (x y): 10 0
Введіть координати C (x y): 0 10
Трикутник коректний

Вийшов трикутник з вершинами: { 0 , 0 },  { 10 , 0 },  { 0 , 10 }


Введіть точку (x y) або будь-що для виходу: 2 2
Точка (2, 2):
  Площа: всередині
  Вектор: всередині

Введіть точку (x y) або будь-що для виходу: -1 -1
Точка (-1, -1):
  Площа: ззовні
  Вектор: ззовні

Введіть точку (x y) або будь-що для виходу: 0 0
Точка (0, 0):
  Площа: на межі
  Вектор: на межі

Введіть точку (x y) або будь-що для виходу: з
Завершення програми
PS D:\university\1\2\OOP\lab2>
```

```
PS D:\university\1\2\OOP\lab2> .\main.exe
Введіть координати A (x y): 0 0
Введіть координати B (x y): 5 5
Введіть координати C (x y): 10 5
Трикутник коректний

Вийшов трикутник з вершинами: { 0 , 0 },  { 5 , 5 },  { 10 , 5 }


Введіть точку (x y) або будь-що для виходу: 5 5
Точка (5, 5):
  Площа: на межі
  Вектор: на межі

Введіть точку (x y) або будь-що для виходу: 0 10
Точка (0, 10):
  Площа: ззовні
  Вектор: ззовні

Введіть точку (x y) або будь-що для виходу: щ
Завершення програми
PS D:\university\1\2\OOP\lab2>
```

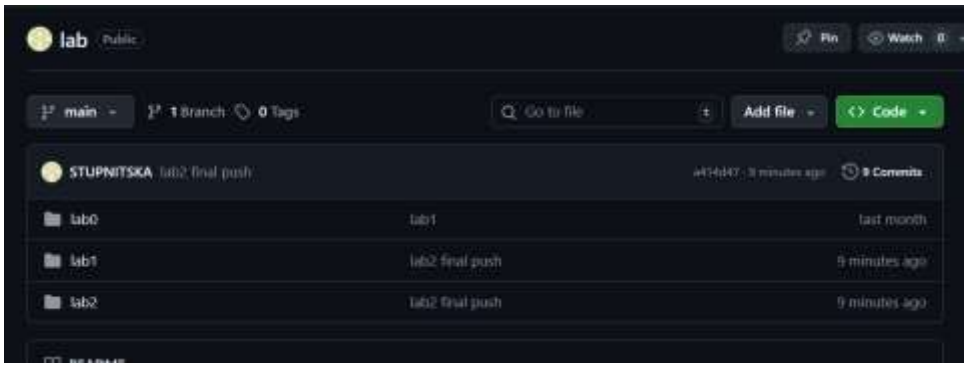
Крок 5. GitHub

Проєкт опубліковано на GitHub.

https://github.com/STUPNITSKA/lab/tree/main/lab2

```
Luminous@DESKTOP-V261ICG MINGW64 /d/university/1/2/OOP (main)
$ git commit -m "lab2 final push"
[main a414d47] lab2 final push
 12 files changed, 579 insertions(+), 69 deletions(-)
 create mode 100644 lab1/.idea/editor.xml
 create mode 100644 lab1/.idea/vcs.xml
 create mode 100644 lab1/lab1.docx
 create mode 100644 lab1/lab1.pdf
 create mode 100644 lab2/main.cpp
 create mode 100644 lab2/main.exe
 create mode 100644 lab2/triangle.cpp
 create mode 100644 lab2/triangle.h

Luminous@DESKTOP-V261ICG MINGW64 /d/university/1/2/OOP (main)
$ git push
Enumerating objects: 24, done.
Counting objects: 100% (24/24), done.
Delta compression using up to 16 threads
Compressing objects: 100% (17/17), done.
Writing objects: 100% (17/17), 744.80 KiB | 27.58 MiB/s, done.
Total 17 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
To https://github.com/STUPNITSKA/lab.git
   29bf6b4..a414d47  main -> main
```



Висновок

У роботі було реалізовано алгоритми перевірки належності точки трикутнику методом площ та векторним методом. Освоєно роботу зі структурами, обробку вводу та використання Git.